SYSTEM DESIGN & ARCHITECTURE DOCUMENT

QR-Based Visitor Management System (VMS)

Version 2.0

Submitted for Internship Evaluation
April 2026

Table of Contents

1. Project Overview & Objectives

The QR-Based Visitor Management System (VMS) is a full-stack web application built as an internship project. It digitizes and automates the end-to-end process of managing visitors at office premises — replacing paper-based registers with a secure, QR-code-driven workflow.

1.1 Problem Statement

Traditional visitor management in offices relies on paper registers, which are prone to data loss, difficult to query, insecure, and impossible to audit in real time. Organizations with multiple branches face additional challenges in maintaining consistency and visibility across locations.

1.2 Solution

VMS provides a centralized digital platform where:

- Admins or guards register visitors with photo capture and form data
- A unique QR code with a configurable expiry is automatically generated
- The QR code is emailed to the visitor and displayed on screen
- Guards scan the QR at entry/exit to record check-in and check-out
- Admins can view dashboards, reports, and manage users across branches
- Super admins have full visibility including an immutable audit log

1.3 Key Objectives

Objective	Description
Digitize visitor flow	**Replace paper registers with digital, searchable records**
QR-based security	**UUID QR tokens prevent guessing; expiry enforces time-bounded access**
Role-based access	**Guard, Admin, Super Admin roles with distinct permissions**
Multi-branch support	**Single deployment serves multiple office locations**
Auditability	**Every action logged — who did what, when, from where**
Modern UI/UX	**React SPA with dark/light mode, responsive design**

2. High-Level System Architecture

VMS follows a classic 3-Tier Architecture pattern separating the system into Presentation, Application, and Data tiers. This separation enables independent scaling, clear responsibility boundaries, and easier maintenance.

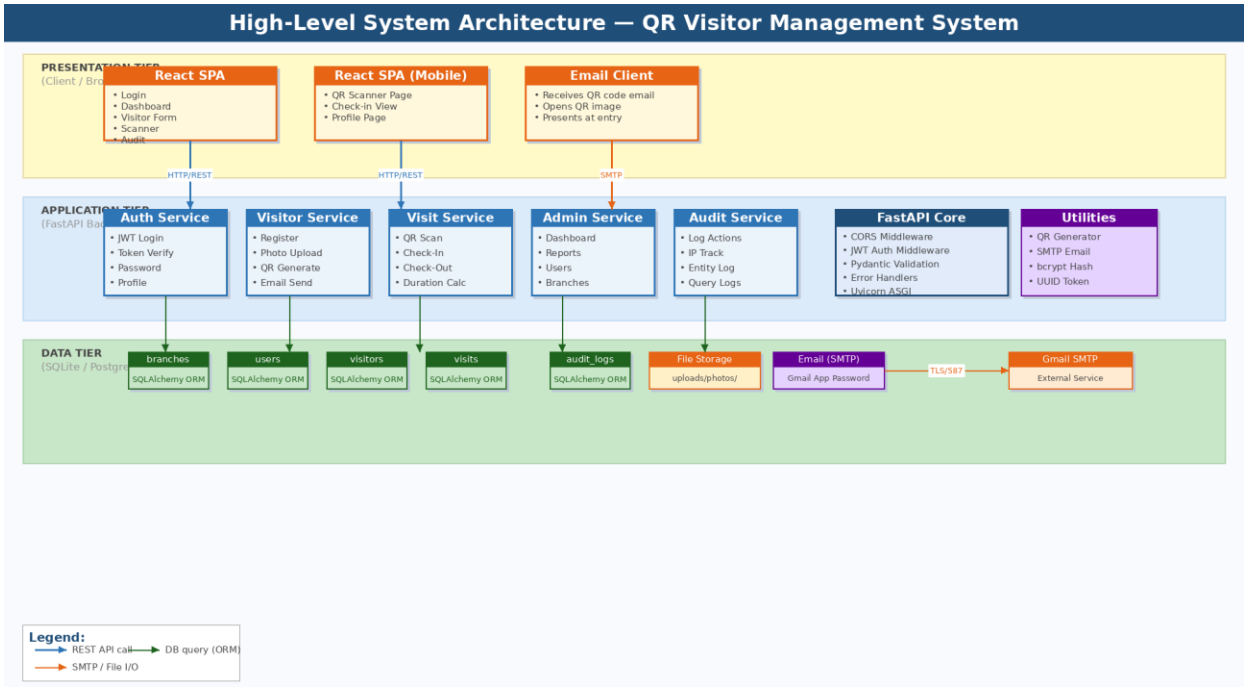

Figure 1: High-Level 3-Tier System Architecture — VMS

2.1 Presentation Tier (Frontend)

Built with React.js as a Single Page Application (SPA). The frontend communicates exclusively with the backend via REST API calls using Axios. It does not directly access the database. Key responsibilities:

- Render all UI pages — login, visitor registration, QR display, scanner, dashboard, audit
- Capture visitor photos using browser webcam via CameraCapture component
- Scan QR codes using the html5-qrcode browser library
- Manage JWT tokens and attach them to every API request via Axios interceptors
- Maintain global auth state via React Context API

2.2 Application Tier (Backend)

Built with FastAPI (Python), this tier is the brain of the system. It processes all business logic, enforces security rules, and orchestrates data access. Key responsibilities:

- Expose RESTful API endpoints organized by domain (auth, visitors, visits, admin)
- Authenticate users via JWT tokens using python-jose
- Implement role-based access control (RBAC) using FastAPI dependency injection
- Generate QR codes as base64 PNG images embedded in visit records
- Dispatch QR code emails via SMTP using Python's smtplib
- Log all significant actions to the audit_logs table

2.3 Data Tier (Database + File Storage)

The persistence layer consists of a relational database (SQLite for development, PostgreSQL for production) accessed via SQLAlchemy ORM, and a local file system for storing visitor photos.

- 5 core tables: branches, users, visitors, visits, audit_logs
- SQLAlchemy models define schema; session factory manages connections
- Visitor photos stored as files under uploads/photos/ directory
- QR code images stored as base64 TEXT in the visits table for fast retrieval

3. Technology Stack

Layer	Technology	Version / Details	Purpose
Frontend	React.js	^18.x	SPA UI framework
	React Router	v6	Client-side routing between pages
	Axios	^1.x	HTTP client + JWT interceptors
	html5-qrcode	^2.x	Browser-based QR code scanner
	Tailwind CSS / Custom CSS		Styling, dark/light mode
	React Context API	Built-in	Global auth & theme state
Backend	FastAPI	^0.100+	Async Python web framework
	Uvicorn	^0.23+	ASGI server for FastAPI
	SQLAlchemy	^2.x	ORM for database access
	Pydantic	v2 (via FastAPI)	Request/response schema validation
	python-jose	^3.x	JWT encode/decode (HS256)
	passlib[bcrypt]	^1.7	Password hashing
	qrcode[pil]	^7.x	QR image generation
	python-multipart	^0.0.6	File upload support
Database	SQLite	3.x (dev)	File-based database for development
	PostgreSQL	14+ (prod)	Production-grade relational database
Email	Gmail SMTP	smtp.gmail.com:587	Email delivery with App Password auth
	smtplib	Python stdlib	SMTP connection management
DevTools	Node.js + npm	^18+	Frontend build tooling
	Python	^3.10+	Backend runtime
	dotenv	^1.x	Environment variable management

4. Component Architecture

The component diagram below shows the internal structure of both the frontend and backend, their individual sub-components, and connections to external services.

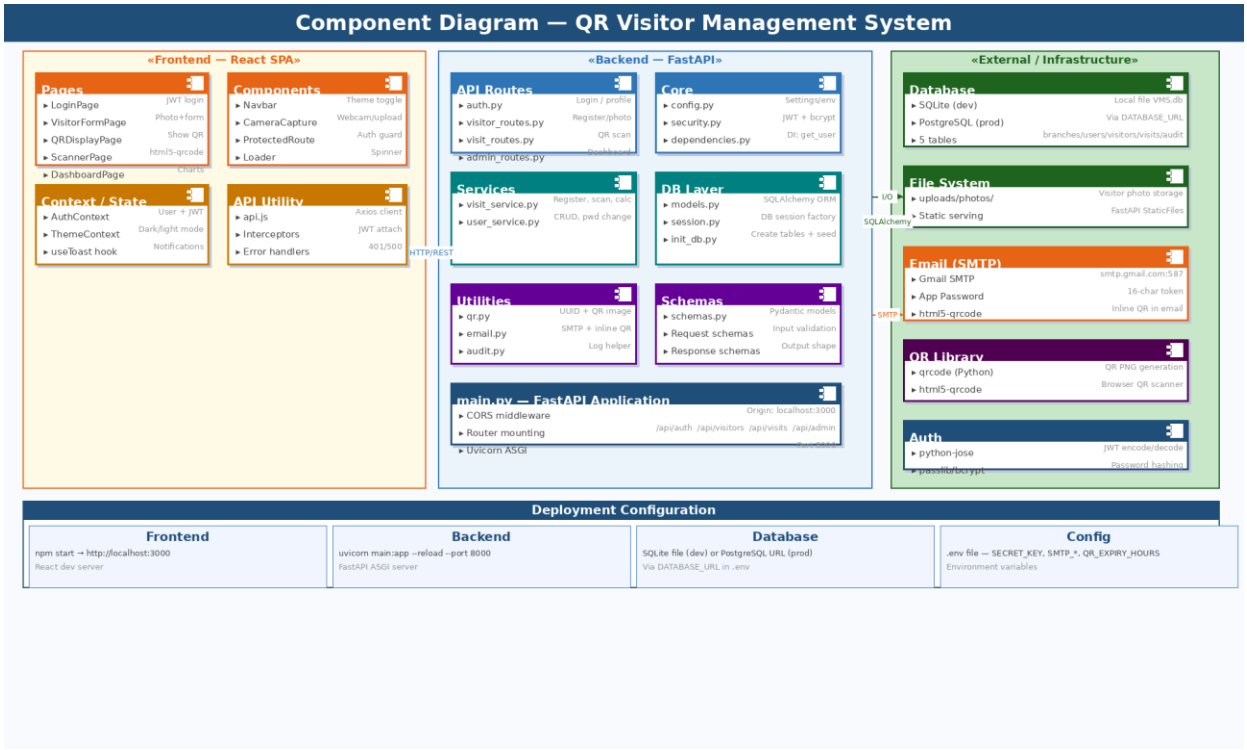

Figure 2: Component Architecture Diagram — Frontend, Backend & External Services

4.1 Frontend Components

4.1.1 Pages

Each page is a top-level React component mapped to a route:

- LoginPage — Email/password form; calls POST /api/auth/login; stores JWT in memory via AuthContext
- VisitorFormPage — Multi-field form with CameraCapture component for photo; calls POST /api/visitors/register
- QRDisplayPage — Shows generated QR from base64 data returned by API; supports download
- ScannerPage — Activates html5-qrcode scanner; calls POST /api/visits/scan/:token
- DashboardPage — Chart-based summary for admins; reads GET /api/admin/dashboard
- AuditPage — Paginated audit log table; restricted to super_admin role

4.1.2 Shared Components & Utilities

- Navbar — Top navigation with role-aware menu items and dark/light mode toggle
- ProtectedRoute — Wrapper that checks AuthContext; redirects to login if unauthenticated or unauthorized
- CameraCapture — Webcam modal using browser MediaDevices API with fallback file upload
- api.js — Axios instance with base URL pointing to FastAPI; interceptor attaches Bearer token to all requests

4.2 Backend Components

4.2.1 API Routes (Routers)

FastAPI routers group endpoints by domain. All routes mounted under /api prefix in main.py:

Router File	Prefix	Key Endpoints
auth.py	**/api/auth**	**POST /login, GET /me, PUT /profile, PUT /change-password**
visitor_routes.py	**/api/visitors**	**POST /register (photo + form), GET / (paginated list)**
visit_routes.py	**/api/visits**	**POST /scan/:token (check-in/out), GET /:id**
admin_routes.py	**/api/admin**	**GET /dashboard, GET /report, GET/POST /users, GET /audit-logs, GET/POST /branches**

4.2.2 Services Layer

- visit_service.py — Handles visitor registration (creates visitor + visit records, generates QR, triggers email), QR scan processing (validates token, checks expiry, updates status, calculates duration)
- user_service.py — CRUD for users, password change (bcrypt verify + re-hash), role assignment, branch assignment

4.2.3 Core / Utilities

- security.py — create_access_token(), verify_token(), get_password_hash(), verify_password() — all auth primitives
- dependencies.py — get_current_user() FastAPI dependency that decodes JWT and returns the user object
- qr.py — Generates UUID token, creates QR PNG via qrcode library, returns base64 data URI
- email.py — Composes HTML email with embedded QR image, connects to SMTP, sends via TLS
- audit.py — log_action() helper that inserts a row into audit_logs with user_id, action, entity, IP

5. Request-Response Flow (Sequence Diagram)

The sequence diagram below traces two critical flows in the system: (1) visitor registration with QR generation and email dispatch, and (2) guard QR scan for check-in or check-out.

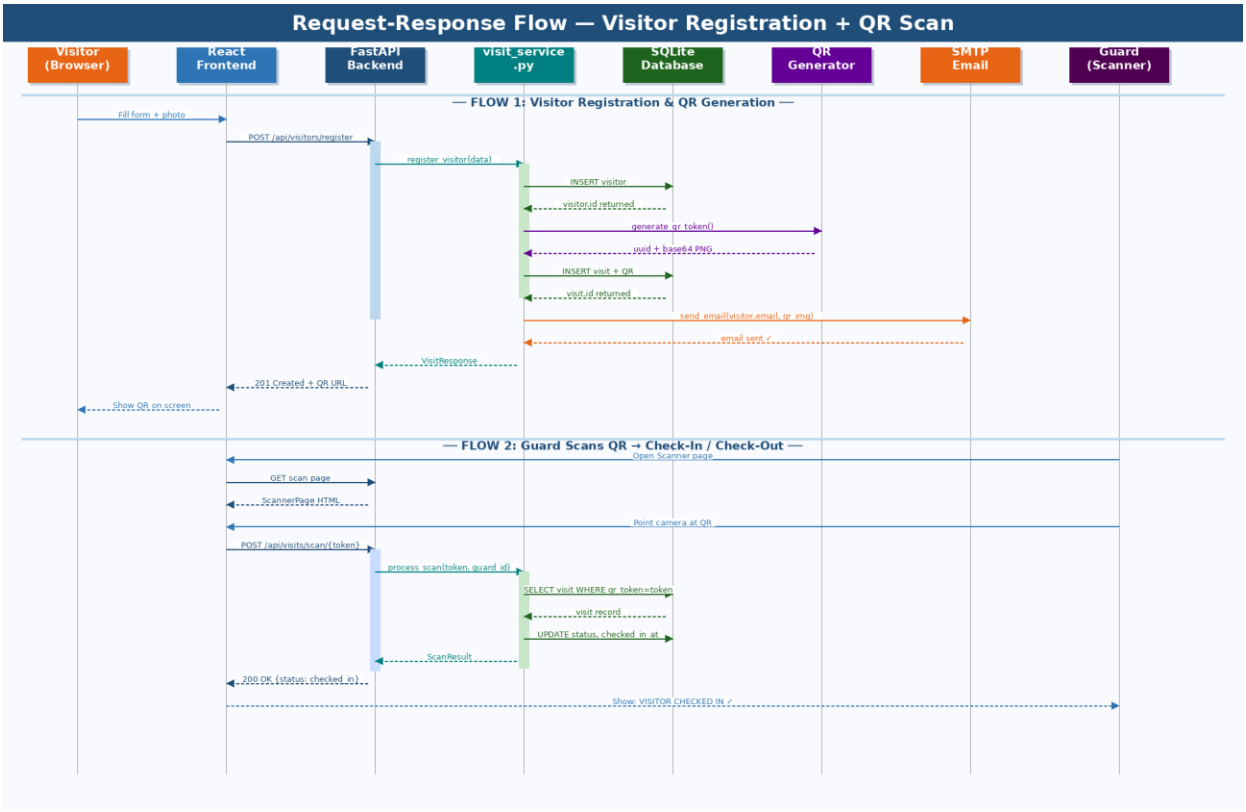

Figure 3: Sequence Diagram — Visitor Registration & QR Scan Flows

5.1 Flow 1 — Visitor Registration

Step-by-step walkthrough:

- Visitor (or guard) fills the form and submits with photo — React calls POST /api/visitors/register
- FastAPI validates the request body via Pydantic schema and calls visit_service.register_visitor()
- Service inserts a visitor record into the visitors table and retrieves the new visitor.id
- qr.py generates a UUID token and creates a base64 QR PNG image
- Service inserts a visit record with qr_token, qr_image, qr_expires, and status=registered
- email.py sends an HTML email with the inline QR image to the visitor's email address
- API returns 201 Created with the visit data including QR URL — React renders the QR on screen

5.2 Flow 2 — Guard QR Scan (Check-In / Check-Out)

- Guard opens the Scanner page; React activates html5-qrcode to read QR from camera
- On successful decode, React calls POST /api/visits/scan/{token}
- FastAPI calls visit_service.process_scan(token, guard_id)
- Service queries visits table: SELECT * WHERE qr_token = token AND qr_expires > NOW()
- If status=registered → set status=checked_in, checked_in_at=NOW(), scanned_by=guard_id
- If status=checked_in → set status=checked_out, checked_out_at=NOW(), duration_mins calculated
- If expired or not found → returns 404 or 410 error

- Response returned to React; guard sees success screen with visitor name and status

6. API Design

The VMS backend exposes a RESTful JSON API. All endpoints follow consistent conventions:

- Base URL: http://localhost:8000/api (development)
- Authentication: Bearer JWT token in Authorization header for all protected routes
- Content-Type: application/json for JSON requests; multipart/form-data for photo upload
- Response format: JSON with consistent shape — data on success, detail on error
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error

6.1 API Endpoint Reference

Method	Endpoint	Auth Required	Role	Description
POST	/api/auth/login	No	—	Login with email+password, returns JWT token
GET	/api/auth/me	Yes	Any	Get current user profile
PUT	/api/auth/profile	Yes	Any	Update name/phone/department
PUT	/api/auth/change-password	Yes	Any	Change password (verify old first)
POST	/api/visitors/register	Yes	Guard+	Register visitor with photo upload
GET	/api/visitors	Yes	Guard+	List all visitors (paginated)
GET	/api/visitors/:id	Yes	Guard+	Get single visitor details
POST	/api/visits/scan/:token	Yes	Guard+	Scan QR — check-in or check-out
GET	/api/visits/:id	Yes	Guard+	Get visit record by ID
GET	/api/admin/dashboard	Yes	Admin+	Summary stats for dashboard
GET	/api/admin/report	Yes	Admin+	Paginated visit report with filters
GET	/api/admin/users	Yes	Super Admin	List all users
POST	/api/admin/users	Yes	Super Admin	Create new user (any role)
DELETE	/api/admin/users/:id	Yes	Super Admin	Deactivate user

Method	Endpoint	Auth Required	Role	Description
GET	/api/admin/branches	Yes	Admin+	List branches
POST	/api/admin/branches	Yes	Super Admin	Create new branch
GET	/api/admin/audit-logs	Yes	Super Admin	Paginated audit log

7. Security Architecture

Security is implemented in multiple layers — each acting as an independent control. The diagram below summarizes the complete security architecture.

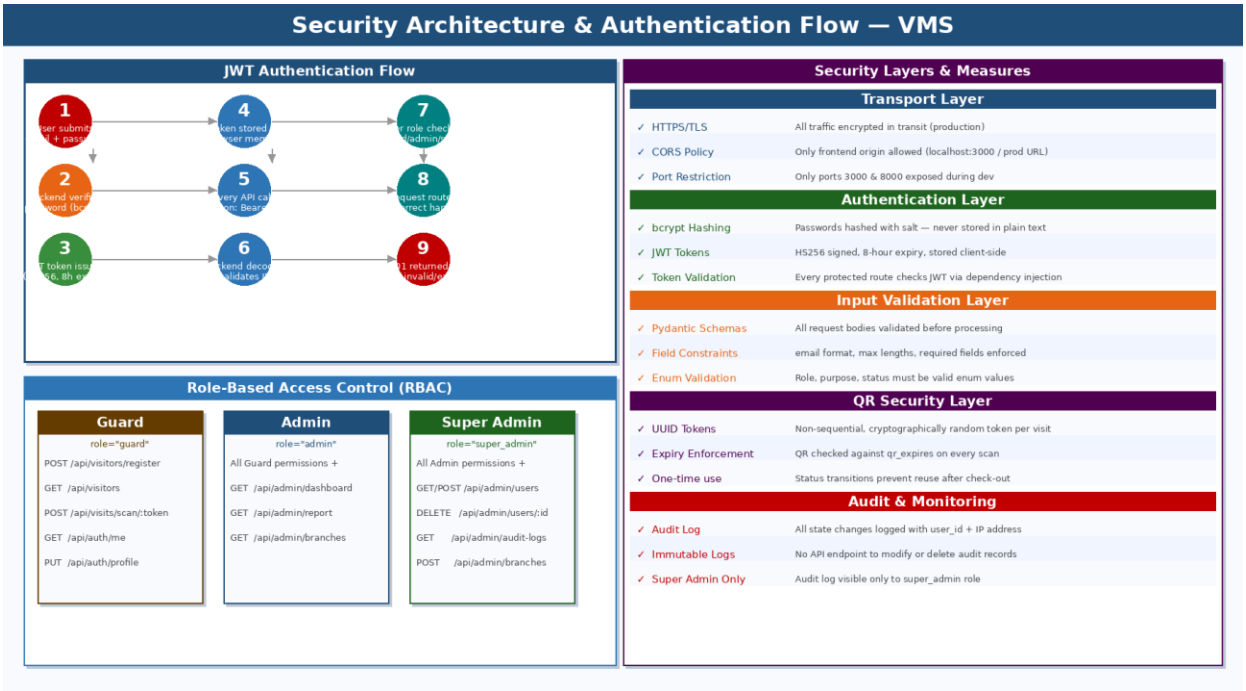

Figure 4: Security Architecture & RBAC — VMS

7.1 Authentication — JWT

VMS uses JSON Web Tokens (JWT) signed with HS256 algorithm for stateless authentication:

- On login, the backend verifies the password using passlib/bcrypt and issues a signed JWT
- Token payload contains user_id, role, branch_id, and exp (expiry) claims
- Token expiry is set to 8 hours (configurable via environment variable)
- React stores the token in-memory via AuthContext — not in localStorage (XSS prevention)
- Every API request attaches the token as Authorization: Bearer <token> via Axios interceptor
- FastAPI's get_current_user() dependency decodes and validates the token on every protected route

7.2 Password Security

- Passwords are hashed using bcrypt with a random salt before storage
- The hashed_password column in users table never stores plain text
- Password comparison uses passlib's verify() — constant-time comparison prevents timing attacks
- Password change endpoint requires the current password to be verified before accepting the new one

7.3 Role-Based Access Control (RBAC)

Three roles with progressively increasing permissions:

Role	Permissions
guard	Register visitors, view visitor list, scan QR for check-in/out, manage own profile
admin	All guard permissions + view dashboard, view reports, view branches
super_admin	All admin permissions + manage users (create/deactivate), manage branches, view audit logs

7.4 QR Code Security

- Each QR token is a UUID v4 — globally unique, non-sequential, unpredictable
- QR tokens are indexed in the database for fast scan lookup
- Every scan validates the token against qr_expires — expired QRs are rejected (410 Gone)
- Status transitions prevent QR reuse: registered → checked_in → checked_out (no reversal)

7.5 Input Validation & Injection Prevention

- All request bodies are validated through Pydantic schemas before reaching service layer
- Field types, lengths, and formats are enforced at schema level
- SQLAlchemy ORM uses parameterized queries — SQL injection is not possible
- File uploads are validated for type and size; stored outside web root with UUID filenames

8. Deployment Architecture

VMS supports two deployment environments: a local development setup and a production cloud/VPS deployment.

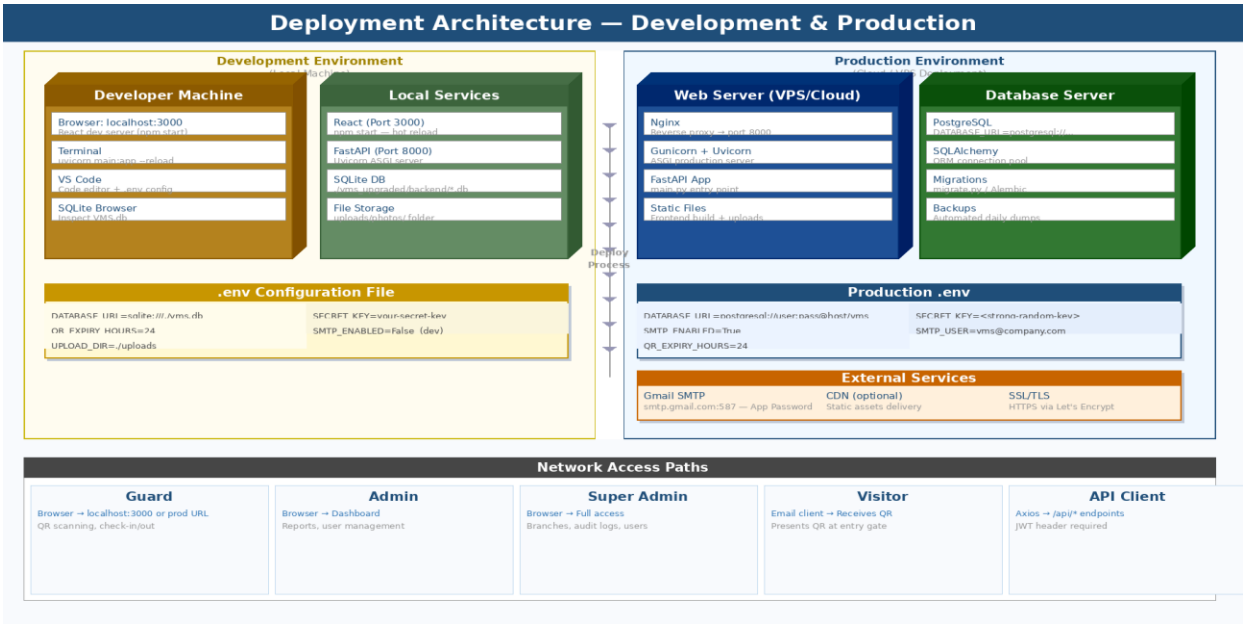

Figure 5: Deployment Architecture — Development & Production

8.1 Development Environment

Component	Command / Config	Details
React Frontend	npm start	Runs on http://localhost:3000 with hot reload
FastAPI Backend	uvicorn main:app --reload --port 8000	Auto-reloads on code changes
Database	DATABASE_URL=sqlite:///./vms.db	SQLite file in backend directory
Environment	.env file	SECRET_KEY, QR_EXPIRY_HOURS, SMTP_ENABLED=False
File Storage	./uploads/photos/	Local directory, served as static files
SMTP	SMTP_ENABLED=False (dev)	Disable email to avoid SMTP setup during dev

8.2 Production Environment

Component	Technology	Details
Web Server	Nginx + Gunicorn	Nginx as reverse proxy; Gunicorn manages Uvicorn workers

Component	Technology	Details
Database	PostgreSQL	DATABASE_URL=postgresql://user:pass@host/vms
Frontend	React build	npm run build → static files served by Nginx or CDN
SSL/TLS	Let's Encrypt	HTTPS enforced; certificate auto-renewed via Certbot
Email	Gmail SMTP	SMTP_ENABLED=True; App Password in .env
Environment	.env (production)	Strong SECRET_KEY; all credentials in environment variables

9. Non-Functional Requirements

NFR Category	Requirement	How VMS Addresses It
Performance	API response < 300ms for CRUD operations	SQLAlchemy indexed queries; QR token indexed for fast scan lookup
Scalability	Support multiple branches and hundreds of users	Multi-branch data model; stateless JWT auth enables horizontal scaling
Security	Protect user data and prevent unauthorized access	bcrypt passwords, JWT auth, RBAC, Pydantic validation, parameterized SQL
Reliability	System available during office hours	Uvicorn ASGI async handling; multiple Gunicorn workers in production
Maintainability	Code organized for future developers	Layered architecture: routes → services → models; .env for all config
Auditability	All changes traceable	Immutable audit_logs with user_id, action, entity, IP, timestamp
Usability	Guards can operate without training	Minimal UI: 3-step visit registration; instant QR display; one-tap scan
Portability	Run on any machine with Python & Node	No OS-specific code; SQLite for dev, PostgreSQL for prod via DATABASE_URL

10. Folder Structure

10.1 Backend (FastAPI)

```
vms_upgraded/backend/ ├── main.py                  ← FastAPI app, CORS, router mounting ├── config.py
← Settings (Pydantic BaseSettings, .env loading) ├── requirements.txt          ← Python dependencies ├──
.env                       ← Environment variables (not committed) ├── app/ │   ├── api/ │   │   ├──
auth.py          ← /api/auth/* endpoints │   │   ├── visitor_routes.py← /api/visitors/* endpoints │   │
├── visit_routes.py  ← /api/visits/* endpoints │   │   └── admin_routes.py  ← /api/admin/* endpoints │
├── db/ │   │   ├── models.py        ← SQLAlchemy ORM models │   │   ├── session.py       ←
SessionLocal, engine, get_db dependency │   │   └── init_db.py       ← create_all() + seed super admin │
├── schemas/ │   │   └── schemas.py       ← All Pydantic request/response models │   ├── services/ │   │
├── visit_service.py ← Business logic for visitor/visit operations │   │   └── user_service.py  ←
Business logic for user management │   └── utils/ │       ├── security.py      ← JWT, bcrypt functions │
├── dependencies.py  ← get_current_user() dependency │       ├── qr.py            ← QR code generation
(UUID + base64 PNG) │       ├── email.py         ← SMTP email with inline QR │       └── audit.py
← log_action() helper └── uploads/     └── photos/              ← Visitor photo files (UUID named)
```

10.2 Frontend (React)

```
vms_upgraded/frontend/ ├── public/                 ← index.html, favicon ├── src/ │   ├── App.js
← Routes, ProtectedRoute, ThemeProvider │   ├── index.js             ← React DOM render entry │   ├──
api.js               ← Axios instance + interceptors │   ├── context/ │   │   ├── AuthContext.js   ←
User state, login/logout │   │   └── ThemeContext.js  ← Dark/light mode │   ├── pages/ │   │   ├──
LoginPage.js │   │   ├── VisitorFormPage.js │   │   ├── QRDisplayPage.js │   │   ├── ScannerPage.js │
│   ├── DashboardPage.js │   │   └── AuditPage.js │   └── components/ │       ├── Navbar.js │       ├──
CameraCapture.js │       ├── Loader.js │       └── ProtectedRoute.js └── package.json            ← npm
dependencies
```

11. Known Limitations & Future Enhancements

11.1 Current Limitations

Limitation	Details
SQLite in Development	**SQLite does not support concurrent writes — not suitable for production under load**
No real-time updates	**Dashboard does not auto-refresh; guard must reload page to see latest visit status**
Email delivery	**SMTP send is synchronous — slow SMTP will delay API response; no retry on failure**
No visitor pre-registration	**Visitors cannot self-register via a public link; registration must be done by staff**
Single file upload	**Only one photo per visitor; no document attachment support**
No SMS / WhatsApp	**QR is sent only via email; visitors without email cannot receive digital QR**

11.2 Future Enhancements

- WebSocket / SSE for real-time dashboard updates without page reload
- Celery + Redis async email queue to decouple SMTP from API response time
- Visitor self-registration via public booking link with host approval workflow
- WhatsApp / SMS QR delivery via Twilio or MSG91 integration
- OTP-based visitor check-in as fallback when QR scan is unavailable
- Docker containerization for consistent deployment across environments
- Alembic database migrations for schema version management
- Mobile app (React Native) for dedicated guard scanner without browser
- Analytics dashboard with CSV/PDF export of visit reports
- Visitor blacklist / blocklist with alert on scan

This document provides the complete system design and architectural reference for the QR-Based Visitor Management System internship project. It should be read alongside the Database Design Document for full technical coverage.